Delete Data with FastAPI

Let's now add a *path operation* to delete a hero.

This is quite straightforward. 😁

Delete Path Operation

Because we want to **delete** data, we use an HTTP `DELETE` operation.

We get a `hero_id` from the path parameter and verify if it exists, just as we did when reading a single hero or when updating it, **possibly raising an error** with a `404` response.

And if we actually find a hero, we just delete it with the **session**.

Python 3.10+

```python
# Code above omitted 👆

def delete_hero(hero_id: int):
    with Session(engine) as session:
        hero = session.get(Hero, hero_id)
        if not hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        session.delete(hero)
        session.commit()
        return {"ok": True}
```

👀 Full file preview

Python 3.10+

```python
from fastapi import FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)


class HeroCreate(HeroBase):
    pass


class HeroPublic(HeroBase):
    id: int


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate):
    with Session(engine) as session:
        db_hero = Hero.model_validate(hero)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(offset: int = 0, limit: int = Query(default=100, le=100)):
    with Session(engine) as session:
        heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
        return heroes


@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int):
    with Session(engine) as session:
        hero = session.get(Hero, hero_id)
        if not hero:
            raise HTTPException(status_code=404, detail="Hero not found")
```

```python
        return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate):
    with Session(engine) as session:
        db_hero = session.get(Hero, hero_id)
        if not db_hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        hero_data = hero.model_dump(exclude_unset=True)
        db_hero.sqlmodel_update(hero_data)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero


@app.delete("/heroes/{hero_id}")
def delete_hero(hero_id: int):
    with Session(engine) as session:
        hero = session.get(Hero, hero_id)
        if not hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        session.delete(hero)
        session.commit()
        return {"ok": True}
```

After deleting it successfully, we just return a response of:

```
{
    "ok": true
}
```

Recap

That's it, feel free to try it out in the interactive docs UI to delete some heroes. 💥

Using **FastAPI** to read data and combining it with **SQLModel** makes it quite straightforward to delete data from the database.